Kinematic Modelling and MATLAB-Based Analysis of Delta Robot Manipulators

Mashedul Yusuf Nastaran Radin Emilian Oxley John Raj Shubham
Department of Mechanical Engineering, University of Bristol, Bristol, UK

1 Introduction

Robot mini-pectors are widely used across industrial, medical and educational domains because of their precision, repeatability and programmability [6]. Common applications include pick-and-place operations, assembly tasks, sorting, dispensing, and laboratory automation. In educational environments, robotic platforms are used to teach students about robot kinematics, motion planning, trajectory generation, control, and system integration.

The Delta Robot Mini-Pector is a desktop-scale manipulator consisting of four revolute joints (J1–J4), forming an equivalent parallelogram structure [6]. It operates in both joint and Cartesian coordinate systems, enabling flexible motion control and trajectory planning. The robot supports multiple motion modes [6] such as point-to-point (PTP), linear (MOVL), and arc-based trajectories, making it suitable for both simple and practical laboratory tasks.

From a classification perspective, the Delta Robot Mini-Pector can be categorized as a parallel manipulator [1, 2, 3, 4, 5]. Compared with serial manipulators, serial manipulators offer a larger workspace and greater flexibility, but may suffer from lower stiffness and accumulated positioning errors. Compared to SCARA robots, the Delta robot provides greater vertical flexibility but may have reduced speed and precision in planar tasks.

The Delta Robot Mini-Pector is particularly suitable for educational use because of its low cost, ease of programmability, and compact footprint [6]. Its modular design and open-source software support rapid prototyping and integration with external laboratory equipment [6], such as conveyors or vision systems. The aim of this project was to develop an analytical and MATLAB-based model of the Delta Robot Mini-Pector, to evaluate its forward kinematics, inverse kinematics, Jacobian, singularities and workspace.

List of Symbols

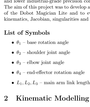
θ_1	= base rotation angle
θ_2	= shoulder joint angle
θ_3	= elbow joint angle
θ_4	= end-effector rotation angle
L_1, L_2, L_3	= main arm link lengths

2 Kinematic Modelling

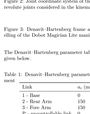
2.1 Robot configuration and assumptions

The kinematic model therefore treats the robot as a 3R serial manipulator, which is describing base rotation, the upper arm, and the forearm, with the end-effector controlling tool orientation.

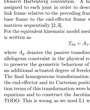
Although the physical robot includes a parallelogram mechanism [6] that helps maintain end-effector orientation, the kinematic model used in this work represents the robot as a serial chain for simplicity and computational efficiency [6]. This passive element is not treated as an additional actuated degree of freedom, but it is considered when defining the forward and inverse kinematic relations.



2.2 Denavit–Hartenberg parameterization



The Denavit-Hartenberg frame assignment used for the kinematic modelling of the Robot Magician Lite manipulator.



The geometric Jacobian is derived from the Denavit-Hartenberg parameters listed in Table 1. Each axis was aligned with the axis of rotation of its corresponding joint, while each x-axis was defined along the common normal between adjacent joint axes. The physical robot has four revolute joints; however, an additional passive transmission was introduced only to represent the parallelogram constraint that keeps the end-effector orientation approximately constant [6]. This passive element was not treated as an extra actuated degree of freedom. The number of actuated degrees of freedom remains four.

Table 1: Denavit-Hartenberg parameters for the Robot Magician Lite

Joint	α_i (rad)	a_i (mm)	d_i (mm)	θ_i
1: Base Rotation	0	90	150	θ_1
2: Rear Arm	150	0	0	θ_2
3: Fore Arm	150	0	0	θ_3
4: End-effector link	0	0	$-d_4$	$\theta_4 - \theta_2 - \theta_3$

The Denavit-Hartenberg convention provides a systematic framework for assigning frame transformations and deriving homogeneous transformations for serial manipulators [2, 3].

2.3 Forward kinematics

The forward kinematics of the Robot Magician Lite were derived using the Denavit-Hartenberg convention. A homogeneous transformation matrix was assigned to each joint in order to describe the position and orientation of each link frame relative to the preceding frame. The overall transformation from the base to the end-effector was then obtained by multiplying the individual joint transformations [2, 3].

Because the passive parallelogram mechanism was not treated as an additional actuated degree of freedom, the kinematic model used in this project, the total transformation was

$$T_{04} = A_1 A_2 A_3 A_4$$

where A_4 denotes the passive transmission introduced to represent the parallelogram constraint in the physical robot. This passive term is included only to preserve the geometric behaviour of the end-effector and does not represent an additional actuated degree of freedom.

The final homogeneous transformation matrix provides both the orientation of the end-effector and its Cartesian position relative to the base frame. The position of the end-effector was extracted from the last column of the transformation matrix and is used in the analytical workspace and singularity analysis.

TODO: This is wrong as we need L1 which is non-zero

$$T_{04} = \begin{bmatrix} r_{00}(\theta_1+\theta_4) & 0 & \sin(\theta_1+\theta_4) & \cos(\theta_1)(L_2\cos(\theta_2)+L_3\cos(\theta_2+\theta_3)) \\ r_{10}(\theta_1+\theta_4) & 0 & -\cos(\theta_1+\theta_4) & \sin(\theta_1)(L_2\cos(\theta_2)+L_3\cos(\theta_2+\theta_3)) \\ 0 & 1 & 0 & h+L_2\sin(\theta_2)+L_3\sin(\theta_2+\theta_3) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.4 Jacobian and singularity analysis

The Jacobian matrix relates joint rates to end-effector motion. In this work, the geometric Jacobian was used to derive the relationship between joint velocities and the linear and angular velocity of the end-effector, while an analytical form was considered for singularity analysis in terms of the selected task variables [6,7]. Jacobian-based analysis is particularly useful for identifying configurations in which the manipulator loses mobility or becomes difficult to control.

Geometric Jacobian

The geometric Jacobian relates the joint velocities to the linear and angular velocity of the end-effector:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = J_v(q)\dot{q}$$

$$J_v(q) = \begin{bmatrix} -r\sin(\theta_1) & -\cos(\theta_1)(L_2\cos(\theta_2)+L_3\cos(\theta_2+\theta_3)) & -L_3\cos(\theta_1)\cos(\theta_2+\theta_3) \\ r\cos(\theta_1) & -\sin(\theta_1)(L_2\cos(\theta_2)+L_3\cos(\theta_2+\theta_3)) & -L_3\sin(\theta_1)\cos(\theta_2+\theta_3) \\ 0 & L_2\cos(\theta_2)+L_3\cos(\theta_2+\theta_3) & -L_3\sin(\theta_2+\theta_3) \end{bmatrix}$$

$$J_\omega(q) = \begin{bmatrix} 0 & -\sin(\theta_1) & -\sin(\theta_1) \\ 0 & \cos(\theta_1) & \cos(\theta_1) \\ 1 & 0 & 0 \end{bmatrix}$$

Where: θ_1 is the base rotation, θ_2 is the shoulder joint angle, θ_3 is the elbow joint angle, θ_4 is the tool rotation, L_2 is the rear arm length, L_3 is the forearm length, h is the base height, r is the radius. Note that L_1 is not included in the Jacobian because it is a constant term that does not affect the velocity mapping.

Analytical Jacobian

The analytical Jacobian satisfies $\dot{x} = J_a(q)\dot{q}$ and is given by

$$J_a(q) = \begin{bmatrix} -r\sin(\theta_1) & -\cos(\theta_1)(-L_2\sin(\theta_2)-L_3\sin(\theta_2+\theta_3)) & -L_3\cos(\theta_1)\sin(\theta_2+\theta_3) \\ r\cos(\theta_1) & -\sin(\theta_1)(-L_2\sin(\theta_2)-L_3\sin(\theta_2+\theta_3)) & -L_3\sin(\theta_1)\sin(\theta_2+\theta_3) \\ 0 & L_2\cos(\theta_2)+L_3\cos(\theta_2+\theta_3) & L_3\cos(\theta_2+\theta_3) \\ 0 & 1 & 1 \end{bmatrix}$$

The base offset h appears in the forward kinematics of z, but it does not appear in the Jacobian because it is a constant term. Its derivative with respect to all joint variables is zero.

Singularities

A singularity occurs when the determinant of the analytical Jacobian becomes zero:

$$\det(J) = L_3 L_2 r \sin(\theta_3) = 0$$

This expression shows that singularities occur when either $\sin(\theta_3) = 0$ or $r = 0$. The condition $\sin(\theta_3) = 0$ corresponds to the arm being fully stretched or fully folded, so that the links become collinear. The condition $r = 0$ corresponds to the wrist lying on the base axis, where the manipulator loses motion authority in certain horizontal directions. Near these configurations, the velocity mapping becomes ill-conditioned and large joint velocities may be required to generate small end-effector motions [6, 7].

2.5 Inverse kinematics

The inverse kinematic problem was solved using a geometric approach, because it provides closed-form solutions and is suitable for real-time implementation [6].

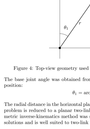


Figure 4: Top-view geometry used to compute the base joint angle θ_1.

The base joint angle was obtained from the target position in the x–y plane:

$$\theta_1 = \operatorname{atan2}\left(\frac{y}{x}\right)$$

The radial distance in the horizontal plane is then $r = \sqrt{x^2 + y^2}$. The remaining problem is reduced to a planar two-link problem in the r–z plane. For a given metric inverse-kinematic method was selected because it provides closed-form solutions and is well suited to real-time planar arm calculations [6].

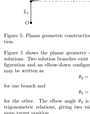


Figure 5: Planar geometric construction used for the inverse kinematics derivation.

Figure 5 shows the planar geometry used to derive the shoulder and elbow angles. The solution was obtained by combining the target position, the base rotation, and an elbow-down configuration. Accordingly, the shoulder angle may be written as

$$\theta_2 = \alpha \pm \beta$$

for one branch and

$$\theta_2 = \alpha - \beta$$

for the other. The elbow angle θ_3 is then obtained from the corresponding trigonometric relation, giving two valid inverse-kinematics solutions for the same target position.

3 Materials and Methods

3.1 Kinematic Modelling Approach

The kinematic model of the Delta Robot Mini-Pector was developed using the Denavit-Hartenberg (DH) convention, which provides a systematic framework for describing the spatial relationship between adjacent links. Each joint was modelled as a revolute joint, and the robot was represented as a serial kinematic chain, despite the physical presence of a four-bar linkage mechanism.

This approach was used because the passive parallelogram mechanism primarily maintains end-effector orientation and does not significantly alter the overall system to be analysed using standard serial manipulator techniques while preserving key geometric behaviour.

3.2 Inverse Kinematics Strategy

A geometric approach was selected for solving the inverse kinematics problem. This method was chosen due to the relatively simple structure of the robot and the availability of clear geometric relationships between joint angles and end-effector position.

The inverse kinematics solution was derived by:

- First solving for the base rotation using $\theta_1 = \operatorname{atan2}(y, x)$
- Projecting the problem into a 2D planar problem in the (r, z) plane
- Applying trigonometric relationships and cosine law to determine θ_2 and θ_3
- Considering both elbow-up and elbow-down configurations

This approach was preferred over numerical methods due to its computational efficiency, analytical traceability and suitability for real-time applications.

3.3 Dynamic modelling approach

The dynamic behaviour of the manipulator was modelled using the standard rigid-body robot dynamics formulation:

$$\tau = M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q)$$

where τ is the vector of joint torques, $M(q)$ is the inertia matrix, $C(q,\dot{q})\dot{q}$ represents Coriolis and centrifugal effects, and $G(q)$ represents gravity.

In the present project, the dynamic equations were incorporated at a conceptual level to support future simulation and control extensions. However, no full dynamic simulation or rigid-body dynamics implementation was limited by the lack of detailed physical parameters such as link masses, inner centres, and inertia tensors from the manufacturer. Therefore, the dynamic analysis was focused on establishing the dynamic form of the equations and explaining how such terms would be included in a MATLAB/Simulink model for future extension.

3.4 MATLAB Implementation

The robot model was implemented in MATLAB using a modular approach. Individual functions were created for the Denavit-Hartenberg transformations, forward kinematics evaluation, inverse kinematics solution, Jacobian computation, and workspace visualisation. This modular organisation was used where appropriate to derive computer representations for the forward kinematics and Jacobian, enabling reusable and easy to modify computational tools.

The implementation was designed to allow multiple configurations to be compared side-by-side, the corresponding joint angles could be computed through inverse kinematics, and the corresponding end-effector pose could be recovered through forward kinematics reconstruction. Additional checks were included to detect unreachable targets and invalid configurations.

3.5 Testing Framework

A structured testing framework was used to validate the correctness of the mathematical model and the MATLAB implementation. Forward kinematics was tested by providing known joint configurations and comparing the resulting end-effector position with the expected geometric result. Inverse kinematics was validated by solving for joint variables from a target Cartesian position and checking whether the forward kinematics could reproduce the original target. Unreachable targets were used to verify singularity detection and to confirm rejection of unreachable targets. Together, these checks ensured internal consistency between the analytical formulation and the computational implementation.

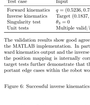
4 Results and Validation

The MATLAB model was validated using forward-kinematics, inverse-kinematics and singularity test cases. Forward kinematics was checked by comparing computed end-effector positions with expected geometric results, and inverse kinematics was verified by reconstructing joint configurations from target Cartesian positions. Additional tests were used to confirm singularity detection and rejection of unreachable targets.

Table 2: Validation of forward and inverse kinematics test results

Test	Expected result	Result
Forward kinematics	$(r, z) = (0.5236, 0.7854, 0.0)$ mm	$(x, y, z) = (0.18437, 0.1596, 0.7854)$
Inverse kinematics	Target $(0.18437, 0.1596, 0.25)$ m	$(x, y, z) = (0.52236, 0.7854, 0.0)$
Reachable target	Solution found	Solution found
Unreachable target	Multiple valid solutions	11/11 tests passed

The validation results show good agreement between the analytical model and the MATLAB implementation. In particular, the consistency between the forward-kinematic output and the inverse-kinematic reconstruction indicates that the position mapping is internally correct. The singularity and unreachable-target tests also confirm that the implementation correctly handles important edge cases within the robot workspace.



The system correctly identifies when a target position lies outside the robot's reachable workspace. In this example, the computed distance λ exceeds the maximum reachable arm length, $\lambda = 0.26$ m, $L_{max} = 0.25$ m, so no inverse solution is found. This demonstrates that the inverse kinematics implementation includes proper workspace validation and prevents physically invalid solutions.

5 Discussion and Conclusion

This project developed and validated a kinematic model of the Delta Robot Mini-Pector using the Denavit-Hartenberg convention, forward and inverse kinematics in MATLAB. Forward and inverse kinematics were derived analytically, the Jacobian was used for differential analysis, and singularities and workspace limitations were identified from the analytical model. The resulting MATLAB implementation was used to reproduce known joint configurations and target-reachable positions, demonstrating internal consistency between the forward and inverse kinematics.

Despite these limitations, the final model provides a robust analytical and computational framework for studying the motion of the Delta Robot Mini-Pector. The analytical formulation and the model is suitable for forward kinematics, inverse kinematics, singularity detection and validation testing in MATLAB. Future work could extend the model to include dynamic simulation, collision checking, trajectory generation, actuator constraints, joint limits and closed-loop control in order to improve realism and practical applicability.

Figure 8: Principal dimensions of the Robot Magician Lite used to support the geometric assumptions of the model. Adapted from [6].

6 References

References

[1] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd, Springer Tracts in Advanced Robotics, 2nd edition, 2017.

[2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, Pearson Education, Upper Saddle River, 3rd edition, 2005.

[3] J. Denavit and R. S. Hartenberg, A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22:215–221, 1955.

[4] Elephant Robotics. Dobot Magician Lite. [Online]. Available: https://cobot.dobot.cc/product-manual/magician-lite/introduction/introduction.html, 2025. 20/Dobot/Lab-Manual.pdf.

[5] MathWorks. Import and apply inverse kinematics to robot arm, 2023. [Online]. Available: https://uk.mathworks.com/help/ros/ug/import-and-apply-inverse-kinematics-to-robot-arm.html.

[6] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, Planning and Control*. Springer London, 2009.

[7] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, Hoboken, 2006.